Exp. No: 6

Date:

A Simple Sign up and Login Mechanism

Aim: -

To create a simple Sign up and Login mechanism and authenticate the user using cookies. The user information can be stored in either MongoDB or MySQL and the server should be built using NodeJS and Express Framework.

Algorithm: -

- Start the Node.js server and establish a connection with the MongoDB database.
- Load the index.html page, which by default shows the login form.
  The user can switch between the login form and the sign-up form using the toggle link.
- The user enters a username and password in the form.When the user clicks the submit button, AngularJS collects the input data.
- If the user selected the login form, AngularJS sends the data to the /api/login endpoint of the backend; otherwise, it sends the data to the /api/signup endpoint.
- The backend receives the request and extracts the username and password from the request body.
- For the sign-up process, the backend checks if the username already exists in MongoDB
- If the username exists, it returns a message stating that the username already exists.
- If the username does not exist, it creates a new user record in the database and returns a message stating that the sign-up was successful.

- For the login process, the backend checks if the username exists in the database.
- If the username is not found or the password does not match, it returns a message stating invalid credentials.
- If the credentials are correct, it returns a message stating that the login was successful.
- The frontend receives the backend response and displays it as a popup message to the user.
- If the login is successful, the frontend stores the username in local storage and navigates the user to dashboard.html.
- On the dashboard page, the user sees a welcome message and a logout button.
- If the user clicks logout, the application clears the stored username from local storage and redirects back to the login page (index.html).

Code: -

Server.js

```
const express = require('express');

const bodyParser = require('body-parser');

const cors = require('cors');

require('./config/db');

const app = express();

app.use(cors());
```

```
app.use(bodyParser.json());

const authRoutes = require('./routes/authRoutes');

app.use('/api', authRoutes);

const PORT = 5000;

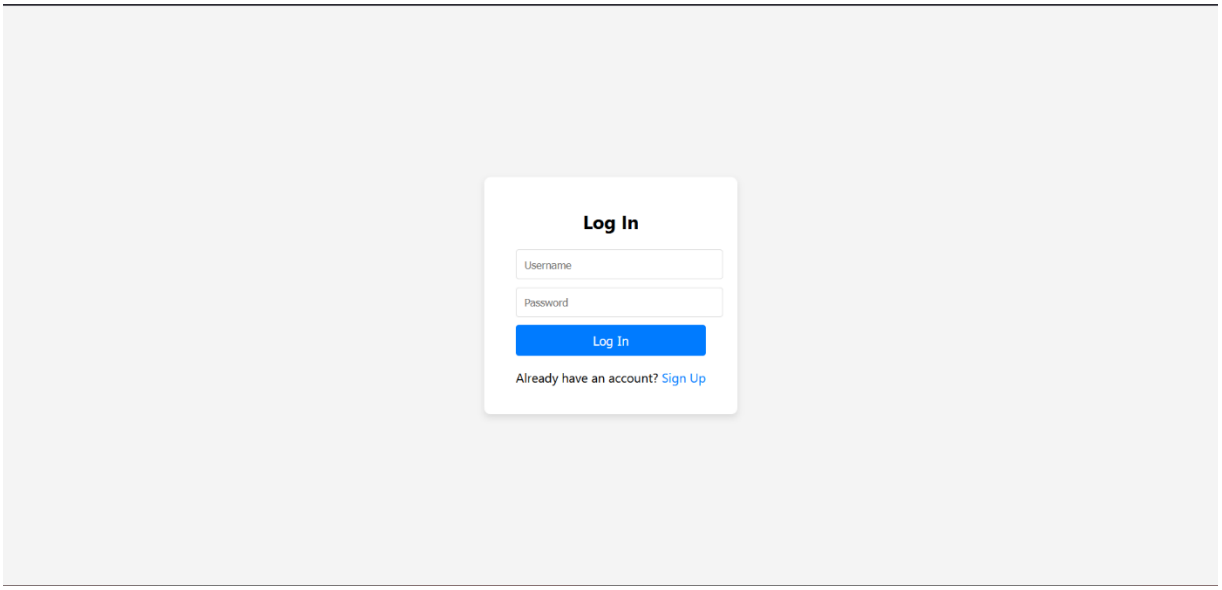
app.listen(PORT, () => console.log(` Server running on ${PORT}`));
```

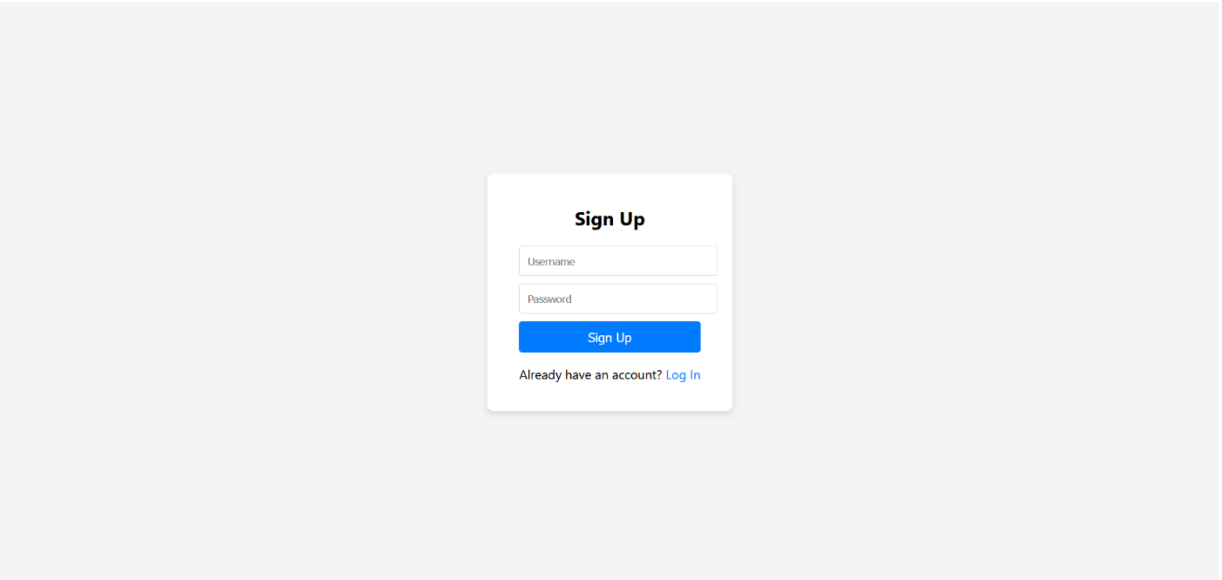
Index.html

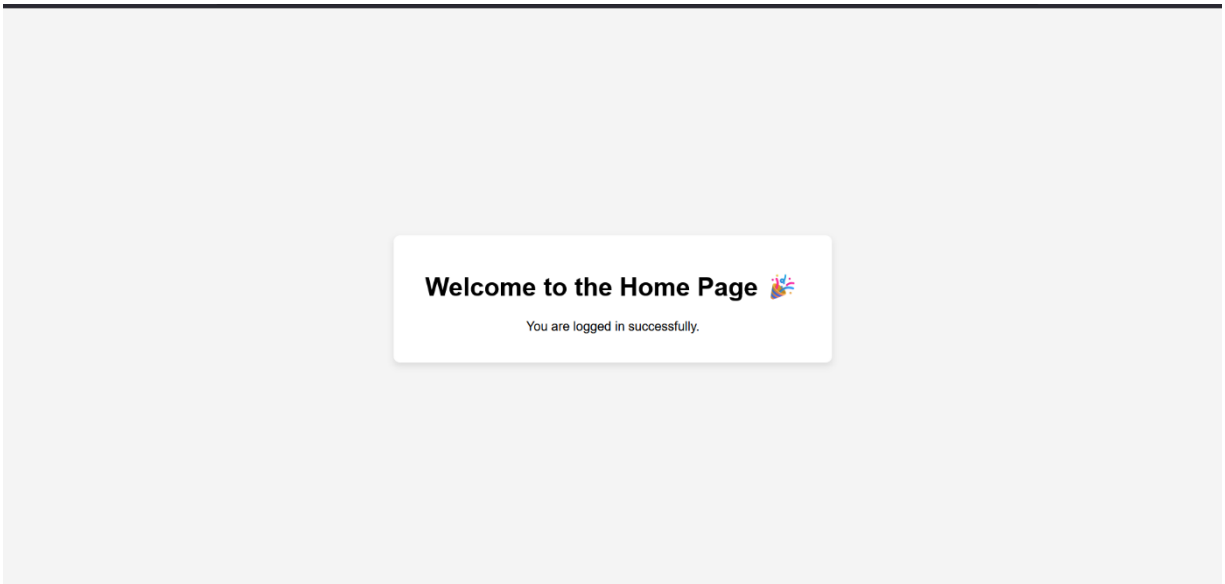
```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Authentication System</title>
   <link rel="stylesheet" href="styles.css">
</head>
<body>
   <div class="container">
      <h2 id="form-title">Sign Up</h2>
      <form id="auth-form">
         <input type="text" id="username" placeholder="Username" required>
         <input type="password" id="password" placeholder="Password" required>
         <button type="submit" id="submit-btn">Sign Up</button>
      </form>
      <p id="message"></p>
      <p>Already have an account? <a href="#" id="toggle-link">Log In</a></p>
   </div>

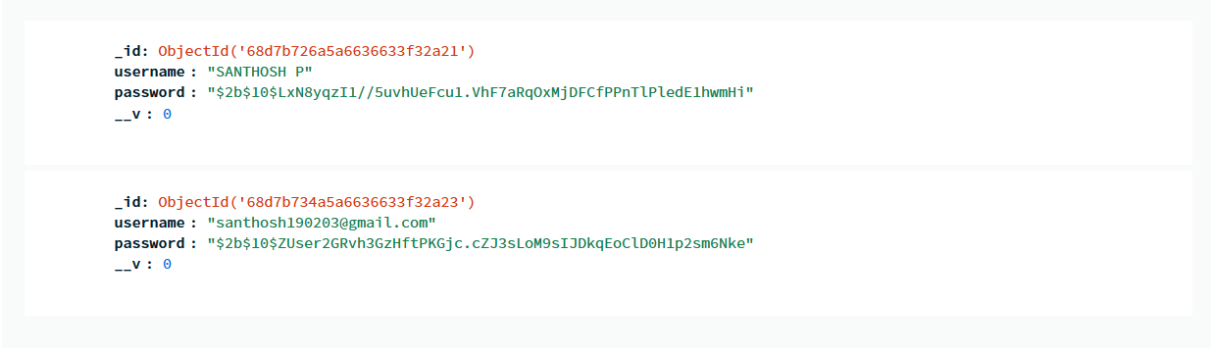
   <script src="app.js"></script>
</body>
</html>
```

OUTPUT:


Log In
Username
Password
Log In
Already have an account? Sign Up


Sign Up
Username
Password
Sign Up
Already have an account? Log In


Welcome to the Home Page 🎉
You are logged in successfully.



COE (30)	
RECORD (10)	
OBS (10)	
VIVA (10)	
TOTAL(60)	

Result: -

Thus, the creation of a simple Sign up and Login mechanism and authenticate the user using cookies. The user information can be stored in either MongoDB or MySQL and the server should be built using NodeJS and Express Framework is completed successfully.

Exp No: 07

Date:

SSH development using virtual box

AIM: -

To create and deploy a virtual machine using a virtual box that can accessed from the host computer using ssh develop.

ALGORITHM: -

STEP 1: Install VirtualBox

- Download VirtualBox from the official website.
- Install it on your host computer.
- Optionally, install VirtualBox Extension Pack for additional features like USB and network support.

STEP 2: Download an OS ISO

- Choose the OS you want to run on the virtual machine (e.g., Ubuntu Server or Desktop).
- Download the ISO file from the official site.

STEP 3: Create a New Virtual Machine

- Open VirtualBox and click "New".
- Enter Name, Type, and Version of the OS.
- Allocate RAM (minimum recommended: 2GB for Ubuntu Desktop, 1GB for Server).
- Create a Virtual Hard Disk (VDI format recommended, dynamically allocated, size ~20GB or more).

STEP 4: Configure VM Settings

- Go to Settings → System:
    - Check Enable EFI if needed.
    - Adjust Processor cores if required.
- Go to Settings → Storage:
    - Attach the OS ISO to the Optical Drive.
- Go to Settings → Network:
    - Change Adapter 1 to Bridged Adapter (or NAT with port forwarding for SSH).

STEP 5: Start the Virtual Machine

- Click Start and boot from the attached ISO.
- Follow the OS installation process:
    - Set username and password.
    - Configure network (DHCP recommended).

STEP 6: Install SSH on the VM

- After OS installation, log into the VM.
- Open terminal and install SSH server:
- sudo apt update
- sudo apt install openssh-server -y
- Enable and start SSH:
- sudo systemctl enable ssh
- sudo systemctl start ssh

STEP 7: Check VM's IP Address

- Get the IP address of the VM:
- ip a
- Note the IP under the network interface (usually eth0 or ens33).

STEP 8: Connect from Host via SSH

- On host computer, open terminal or use an SSH client like PuTTY (Windows).
- Connect using:
- ssh username@VM_IP_Address
- Enter the password set during VM setup.

STEP 9: Optional Configuration

- For NAT network: configure Port Forwarding in VirtualBox settings:
    - Host Port: 2222, Guest Port: 22
    - Connect using ssh username@localhost -p 2222
- Verify SSH connection works consistently.

PROGRAM: -

Vagrantfile code

```
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/focal64"   # Ubuntu 20.04


  # Port forwarding
  config.vm.network "forwarded_port", guest: 22, host: 2222  # SSH
  config.vm.network "forwarded_port", guest: 3001, host: 3001 # Node.js
  config.vm.network "forwarded_port", guest: 4200, host: 1234 # Angular
  config.vm.network "forwarded_port", guest: 27017, host: 27017 # MongoDB

  # Provision software automatically
  config.vm.provision "shell", inline: <<-SHELL
  sudo apt-get update -y
    curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
    sudo apt-get install -y nodejs build-essential
    sudo apt-get install -y mongodb
  sudo systemctl enable mongodb
  sudo systemctl start mongodb
  sudo npm install -g @angular/cli
  SHELL
End
```

Server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const app = express();
app.use(cors());
app.use(express.json());
// MongoDB connection
mongoose.connect("mongodb://127.0.0.1:27017DB", {
  useNewUrlParser: true,
  useUnifiedTopology: true,
});
const User = mongoose.model("User", new mongoose.Schema({ name: String }));
// Routes
app.get("/", (req, res) => res.send("Backend is running!"));
app.post("/users", async (req, res) => {
  const user = new User(req.body);
  await user.save();
  res.json(user);
});
app.get("/users", async (req, res) => res.json(await User.find()));
// Start server
```

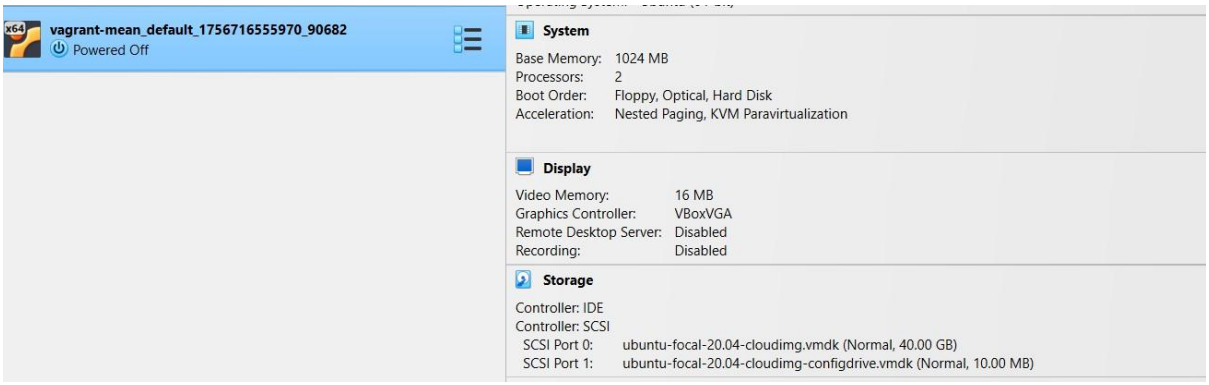
app.listen(3000, () => console.log("Backend running on port 3000"));

OUTPUT: -

```
Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
  → Local:   http://localhost:4200/
  → Network: http://192.168.56.1:4200/
  → Network: http://192.168.137.223:4200/
  → press h + enter to show help
```

```
(node:15440) [MONGODB DRIVER] Warning: useNewUrlParser is a deprecated option: useNewUrlParser has no effect since No
oved in the next major version
(Use `node --trace-warnings ...` to show where the warning was created)
(node:15440) [MONGODB DRIVER] Warning: useUnifiedTopology is a deprecated option: useUnifiedTopology has no effect si
be removed in the next major version
Backend running on port 3000
```

```
PS E:\Full_Stack\Exp_7\vagrant-mean> vagrant ssh
>>
VM must be running to open SSH connection. Run `vagrant up`
to start the virtual machine.
```



COE (30)	
RECORD (10)	
OBS (10)	
VIVA (10)	
TOTAL (60)	

RESULT: -

Thus, the implementation of creating and deploy a virtual machine using a virtual box that can accessed from the host computer using ssh develop is completed successfully

Exp No: 08

Date:

DEPLOY A NODEJS PING USING THE NODEJSIMAGE

AIM: -

To create a docker container that will deploy a nodejs ping server using the nodejsimage.

ALGORITHM

Step 1: Import Required Modules

- Load the `express` library, which simplifies handling HTTP requests.

```
const express = require("express");
```

Step 2: Initialize the Express Application

- Create an Express application object to set up routes and handle server operations.

```
const app = express();
```

Step 3: Define the Server Port

- Use an environment variable `PORT` if available, otherwise default to `3000`.

```
const PORT = process.env.PORT || 3000;
```

Step 4: Create the Root Route (/)

- Define what happens when the user visits `http://localhost:3000/`.
- Send an HTML response (welcome message and link to `/ping`).

```
app.get("/", (req, res) => {
  res.send(`
    <h1>Welcome to the Node.js Ping Server 🚀</h1>
    <p>Try accessing <a href="/ping">/ping</a> to get a pong response.</p>
```

```
  `);
});
```

Algorithm for this route:

1. User enters `/` in the browser.
2. Server receives `GET /` request.
3. Server responds with an HTML message.
4. Browser displays the HTML page.

Step 5: Create the Ping Route (`/ping`)

- Define what happens when the user visits `http://localhost:3000/ping`.
- Send a JSON response `{ message: "pong" }`.

```
app.get("/ping", (req, res) => {
  res.json({ message: "pong" });
});
```

Algorithm for this route:

1. User enters `/ping` in the browser or sends a GET request.
2. Server receives `GET /ping` request.
3. Server creates a JSON object `{ "message": "pong" }`.
4. Server sends the JSON response back to the client.
5. Browser (or client tool like Postman/curl) displays the JSON output.

Step 6: Start the Server

- Start listening on the defined port (3000).
- Print a message in the console.

```
app.listen(PORT, () => {
  console.log(`Ping server running on port ${PORT}`);
});
```

1. The server binds to port `3000`.
2. The process keeps running, waiting for client requests.

3. Whenever a request comes (either `/` or `/ping`), the correct route handler executes.

Step 7: Dockerize the Application

- Create a `Dockerfile` with the following steps:
    1. **Base Image**: Use official `node:18`.
    2. **Working Directory**: Set `/app` as the working directory.
    3. **Copy Files**: Copy `package.json` and `package-lock.json` into the container.
    4. **Install Dependencies**: Run `npm install` inside the container.
    5. **Copy Code**: Copy remaining files (like `server.js`).
    6. **Expose Port**: Expose port `3000` so Docker can map it.
    7. **Start App**: Run `node server.js`.

Step 8: Build and Run the Docker Image

1. Build the image:
2. `docker build -t node-ping-server .`
3. Run the container:
4. `docker run -p 3000:3000 node-ping-server`
5. Test in browser:
    - `http://localhost:3000/` → HTML Welcome Page
    - `http://localhost:3000/ping` → `{"message":"pong"}`

PROGRAM: -

Server.js

```
const express = require("express");

const app = express();

const PORT = process.env.PORT || 3000;
```

```
// Root route
app.get("/", (req, res) => {
  res.send(`
    <h1>Welcome to the Node.js Ping Server 🚀</h1>
    <p>Try accessing <a href="/ping">/ping</a> to get a pong response.</p>
  `);
});
// Ping route
app.get("/ping", (req, res) => {
  res.json({ message: "pong" });
});
app.listen(PORT, () => {
  console.log(`Ping server running on port ${PORT}`);
});
```

DockerFile

```
FROM node:18
WORKDIR /app COPY
package*.json ./ RUN npm
install
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

OUTPUT: -

```
This message is shown once a day. To disable it please create the
/home/siva/.hushlogin file.
siva@LAPTOP-JHPP8PKH:/mnt/c/Users/SIVA$ mkdir node-ping-server
cd node-ping-server
npm init -y
npm install express
Wrote to C:\Users\SIVA\node-ping-server\package.json:

{
  "name": "ping-server",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}



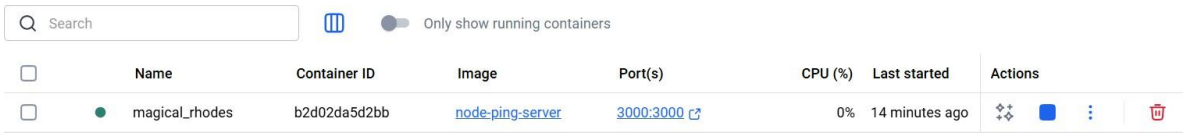
added 67 packages, and audited 68 packages in 4s

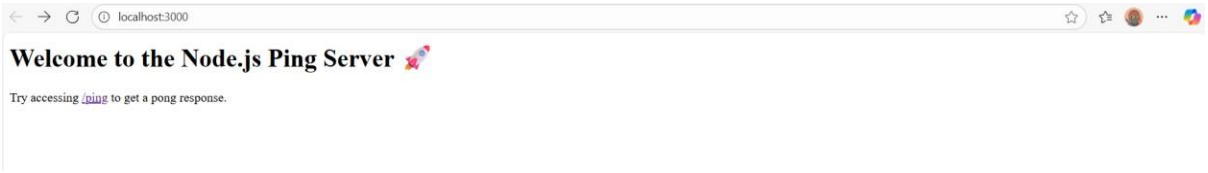
14 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
siva@LAPTOP-JHPP8PKH:/mnt/c/Users/SIVA/node-ping-server$ nano server.js
```

```
siva@LAPTOP-JHPP8PKH:/mnt/c/Users/SIVA/node-ping-server$ docker --version
Docker version 28.3.3, build 980b856
siva@LAPTOP-JHPP8PKH:/mnt/c/Users/SIVA/node-ping-server$ docker build -t node-ping-server
[+] Building 174.5s (11/11) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 148B
 => [internal] load metadata for docker.io/library/node:18
 => [auth] library/node:pull token for registry-1.docker.io
 => [internal] load .dockerignore
 => => transferring context: 2B
```

```
siva@LAPTOP-JHPP8PKH:/mnt/c/Users/SIVA/node-ping-server$ docker run -p 3000:3000 node-ping-server
Ping server running on port 3000
```





COE (30)	
RECORD (10)	
OBS (10)	
VIVA (10)	
TOTAL (60)	

RESULT: -

Thus, the implementation of docker container that will deploy a nodejs ping server using the nodejsimage is executed successfully.

Exp No: 9

Date:

Quiz Application using MEAN

AIM: -

To build an online MCQ quiz app. The questions and options should be fetched based on the chosen topic from a NodeJS server. The questions can be stored in a JSON file in the backend. Once the user has answered the questions, the frontend must send the chosen options to the backend and the backend must identify the right answers and send the score back to the front end. The frontend must display the score in a separate neatly designed page.

ALGORITHM:

Frontend

STEP 1: Design the initial landing page to prompt the user to choose a quiz topic.

STEP 2: On topic selection, the frontend sends a GET request to the backend with the chosen topic.

STEP 3: After receiving the questions from the backend, the frontend renders the quiz interface, displaying questions and multiple-choice options one by one.

STEP 4: The frontend keeps track of the user's selected answer for each question.

STEP 5: Once all questions are answered, the frontend packages the user's responses into a JSON object and sends it to the backend via a POST request.

STEP 6: The frontend receives the score from the backend's response and displays it on a new, dedicated score page.

Backend

STEP 1: Set up a Node.js server to listen for incoming requests from the frontend.

STEP 2: Create a GET endpoint that, when given a topic, reads the corresponding JSON file containing the questions from the file system.

STEP 3: The backend sends the question data back to the frontend as a JSON response.

STEP 4: Create a POST endpoint to receive the user's answers.

STEP 5: The backend compares the user's submitted answers with the correct answers stored in the JSON file.

STEP 6: The backend calculates the final score based on the comparison.

STEP 7: The backend sends the calculated score back to the frontend in a JSON response.

PROGRAM: -

Seed.js

```
const mongoose = require('mongoose');

const QuestionSchema = new mongoose.Schema({

  question: String,

  options: [String],

  answer: [String]

});

const Question = mongoose.model('Question', QuestionSchema);

mongoose.connect('mongodb+srv://sivak24mca:sivakmca@clusterquiz.mx48s6c.mongodb.net/?retryWrites=true&w=majority&appName=Clusterquiz', { useNewUrlParser: true, useUnifiedTopology: true })

  .then(() => Question.insertMany([

{

    question: "What is the capital of France?",

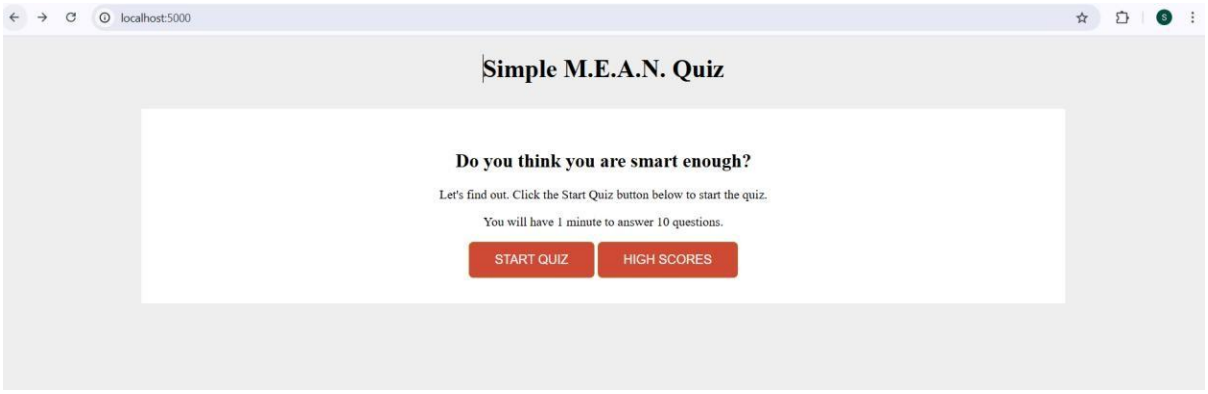
    options: ["Paris", "London", "Berlin", "Rome"],

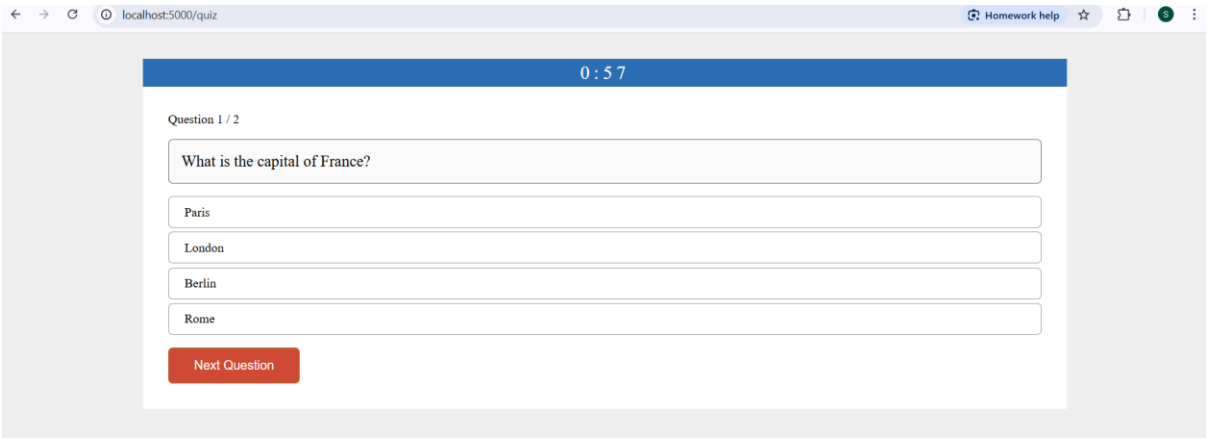
    answer: ["Paris"]

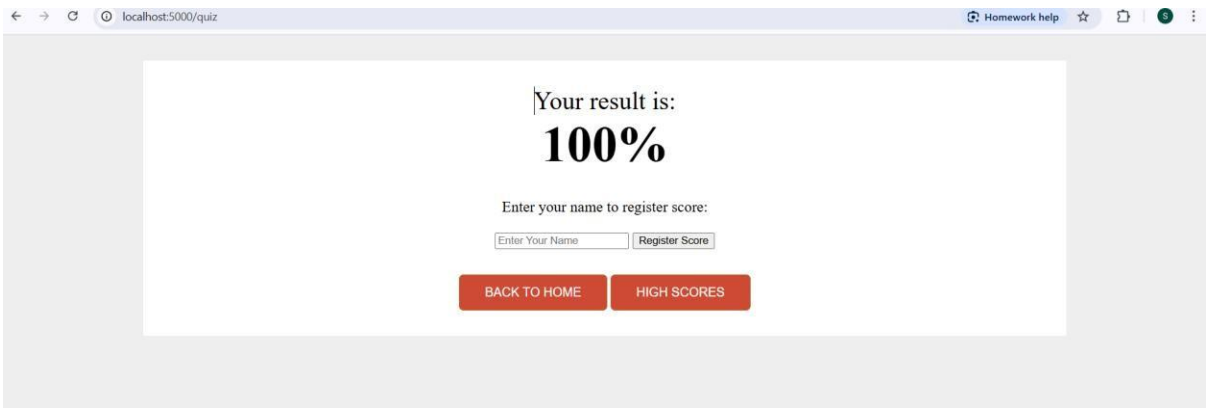
  },  {
```

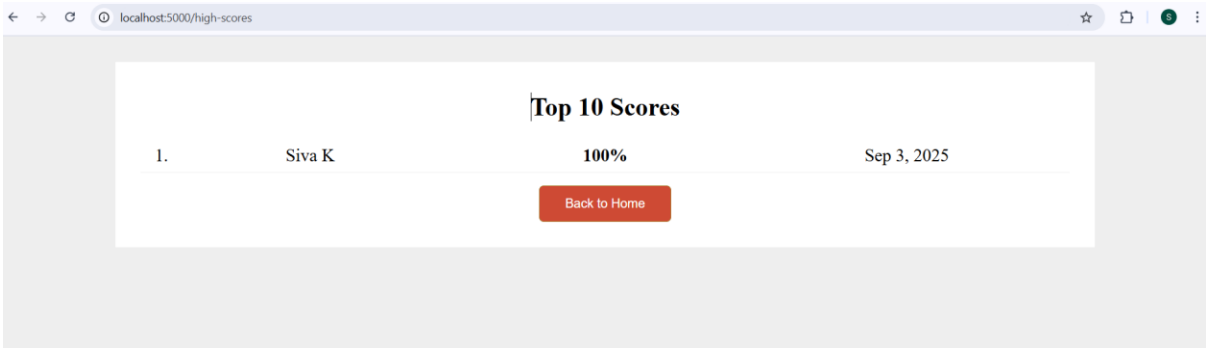
```
    question: "Which language is used with MEAN stack?",
    options: ["Python", "JavaScript", "Java", "C#"],
    answer: ["JavaScript"]
  }
]))
.then(() => { console.log("Seeded!"); mongoose.disconnect(); })
.catch(console.error);
```

OUTPUT: -










```
(node:16172) [MONGOOSE] DeprecationWarning: Mongoose: the `strictQuery` option will be switched back
to `false` by default in Mongoose 7. Use `mongoose.set('strictQuery', false);` if you want to prepare
 for this change. Or use `mongoose.set('strictQuery', true);` to suppress this warning.
(Use `node --trace-deprecation ...` to show where the warning was created)
Server started on port: 5000
Connected to database mongodb+srv://sivak24mca:sivakmca@clusterquiz.mx48s6c.mongodb.net/?retryWrites=
true&w=majority&appName=Clusterquiz
```

COE (30)	
RECORD (10)	
OBS (10)	
VIVA (10)	
TOTAL (60)	

RESULT: -

Thus, the implementation of building the MCQ quiz application is successfully executed.

Exp No: 10

Date:

BUILD A BLOG WEBSITE

AIM: -

To Build a blog website where you can add blog posts through a simple admin panel and the users can view the blog posts. The contents of the blog posts can be stored in either MongoDB or MySQL database. The home page should contain the titles of the blog post and the full post can be viewed by clicking the title. Frontend can be built either using AngularJS or through template engines served by the NodeJS server.

ALGORITHM: -

1. **Start the Application**
    - Run npm start.
    - Node.js executes app.js.
    - Express server starts on http://localhost:3000.
2. **Initialize Server and Middleware**
    - Import required packages (express, mongoose, body-parser, path, ejs).
    - Configure middleware for parsing request bodies (body-parser.urlencoded).
    - Set view engine to ejs so .ejs templates can be rendered.
    - Define public folder for static files (CSS, images, JS).
3. **Connect to MongoDB**
    - Import mongoose.
    - Load Post model (title, content, createdAt).
    - Connect to MongoDB using mongoose.connect() with the given URL.
    - Handle connection success or failure with console messages.
4. **Define Routes**
    - / (Home Route): Fetch all posts from MongoDB using Post.find(). Render index.ejs with a list of posts.
    - /post/:id: Fetch a single post by its _id from MongoDB using Post.findById(). Render post.ejs to display the post details.

- /admin: Render admin.ejs which contains a form for adding a new blog post.
- /admin/add (POST): Accept form submission, create a new Post document with title, content, and save it to MongoDB. Redirect to /.

5. **Handle Views (EJS Templates)**
   - **layout.ejs**: Common layout with <header>, <footer>, and navigation menu.
   - **index.ejs**: Displays a list of blog posts (title + snippet + link to "Read More").
   - **post.ejs**: Displays a single blog post with title, publish date, and content (styled).
   - **admin.ejs**: Displays form to add a new blog post (title + content fields).
6. **Adding a Post (Admin Panel)**
   - Admin opens /admin in browser.
   - Fills form with post title and content.
   - Submits form → request goes to /admin/add.
   - Server processes request, creates a new post in MongoDB, and redirects to /.
7. **Viewing Posts (User Side)**
   - User opens /.
   - Server fetches all posts from MongoDB.
   - Renders index.ejs showing blog titles as clickable links.
   - User clicks a post → request goes to /post/:id.
   - Server fetches that post by ID and renders post.ejs with details.
8. **Error Handling**
   - If MongoDB connection fails, log an error.
   - If post.id doesn't exist, send a "Post not found" message.
   - If routes don't match, send 404 page.
9. **Enhancements** (you already applied)
   - Improved blog design in post.ejs with CSS, shadows, and layout.
   - Fixed issues with layout not defined by switching to normal HTML structure instead of express-ejs-layouts.
10. **Final Execution**
    - Server continuously listens on port 3000.
    - Admin can add posts.
    - Users can read posts.
    - MongoDB stores all posts permanently.

PROGRAM: -

App.js

```
const express = require('express');

const mongoose = require('mongoose');

const bodyParser = require('body-parser');

const indexRoutes = require('./routes/index');

const adminRoutes = require('./routes/admin');

const app = express();

// Connect to MongoDB

mongoose.connect('mongodb+srv://sivak24mca:sivakmca@cluster0.zdbjk50.mongodb.net/?
        retryWrites=true&w=majority&appName=Cluster0', {

  useNewUrlParser: true,

  useUnifiedTopology: true

}).then(() => console.log('MongoDB connected'))

  .catch((err) => console.log('MongoDB connection error:', err));

// Middleware

app.set('view engine', 'ejs');

app.use(bodyParser.urlencoded({ extended: true }));

// Routes

app.use('/', indexRoutes);

app.use('/admin', adminRoutes);
```

```
// Start server

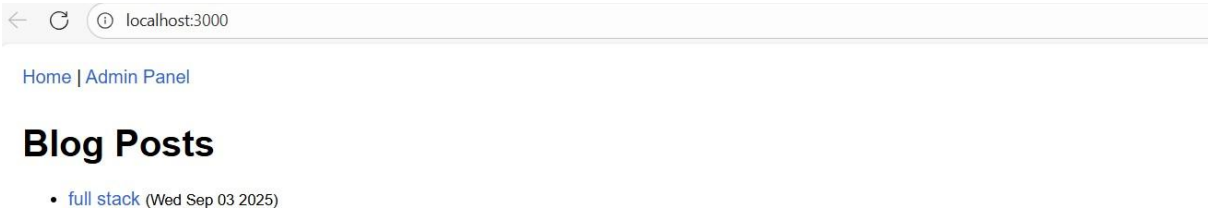
const PORT = 3000;

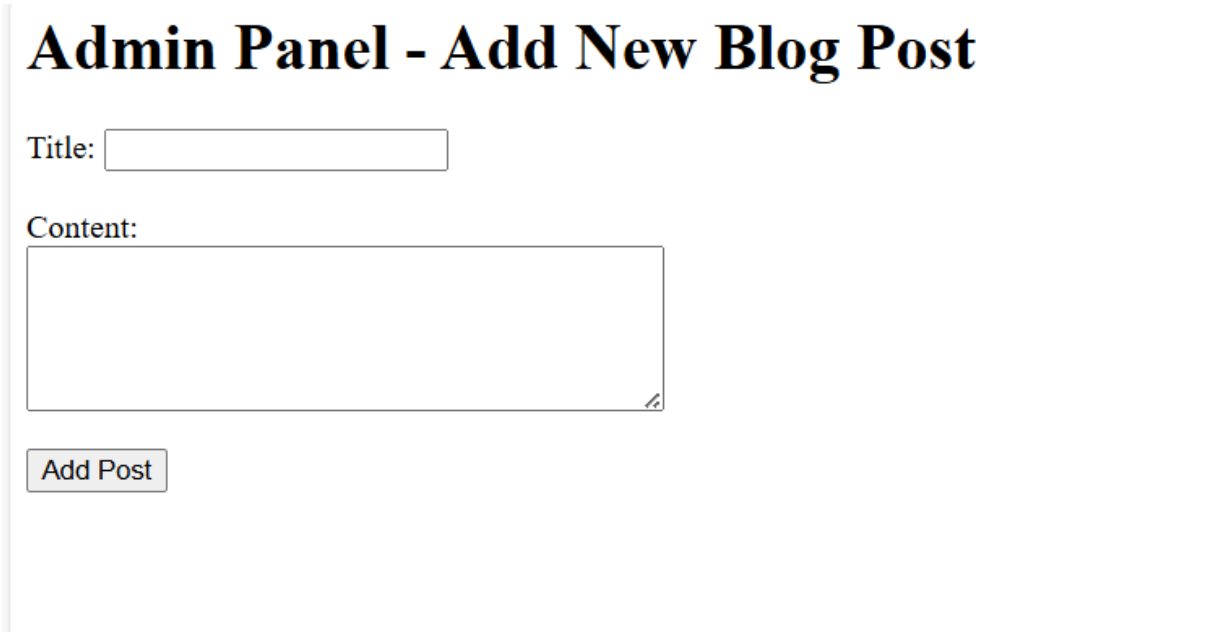
app.listen(PORT, () => {

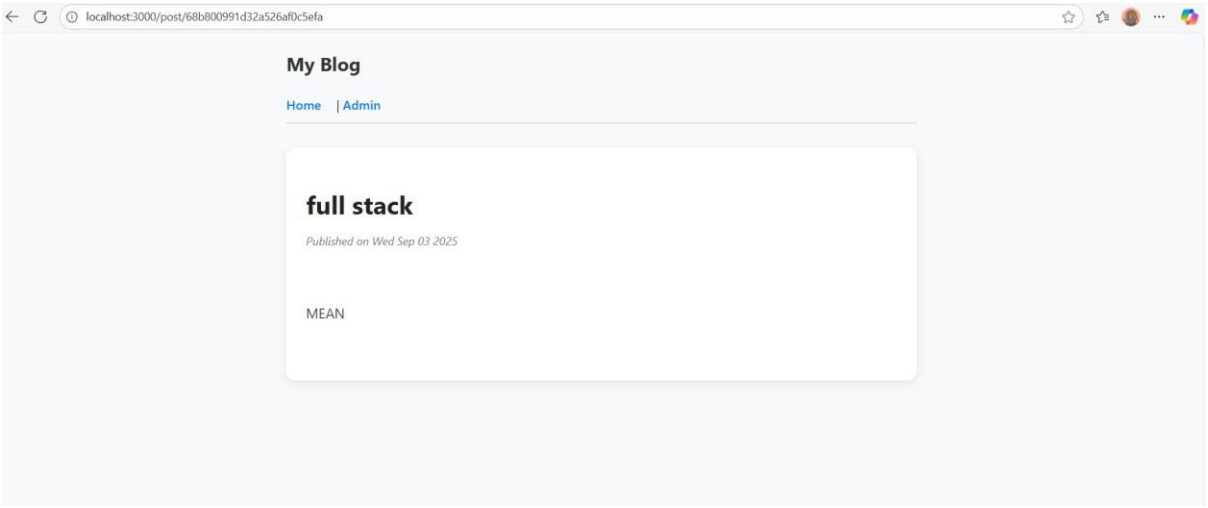
  console.log(`Server started on http://localhost:${PORT}`);

});
```

OUTPUT: -







COE (30)	
RECORD (10)	
OBS (10)	
VIVA (10)	
TOTAL (60)	

RESULT: -

Thus, the implementation of blog creation using MEAN stack with admin panel is successfully executed.

Exp No: 11

Date:

SIMPLE CALCULATOR USING ANGULAR JS

AIM: -

To build a simple calculator app with AngularJS. The user should be able to add numbers and operations to the app by clicking on buttons, just like you would do in a mobile phone. The moment the operation and the two operations are defined, the answer should be displayed.

ALGORITHM: -

STEP 1: Initialize Variables. Set up variables to manage the calculator's state. You'll need a variable to store the value currently on the display (display), a variable to hold the first number of the calculation (firstNumber), and a variable to store the selected operation (operator). Initially, display should be '0', and the other two variables should be null.

STEP 2: Handle Number Input. When a user clicks a number button, a function should be called that appends the new digit to the current display value.

STEP 3: Handle Operator Input. When a user clicks an operator button (+, -, *, /), a function should be called that performs the following actions:

- Stores the current value of display into the firstNumber variable.
- Stores the selected operator into the operator variable.
- Clears the display to allow the user to enter the second number.

STEP 4: Handle Calculation. When the user clicks the equals button (=), a function should be triggered to perform the final calculation. This function should:

- Retrieve the second number from the display.
- Use a conditional statement (like a switch or if/else if) to perform the correct operation (+, -, *, /) using firstNumber and the second number.
- Update the display with the final result.

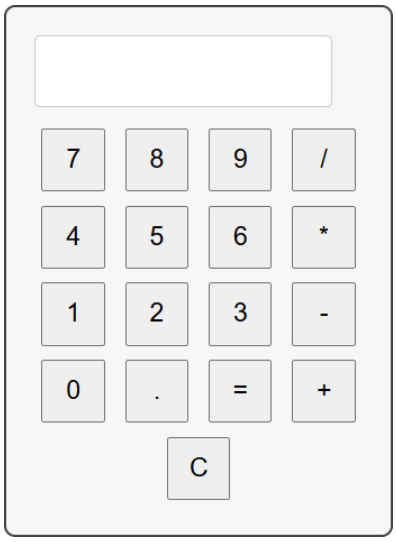
STEP 5: Handle Clear. When the 'C' button is clicked, a function should reset all variables (display, firstNumber, operator) to their initial state.

PROGRAM: -

```
<!DOCTYPE html>
<html ng-app="calculatorApp">
<head>
<meta charset="UTF-8">
<title>AngularJS Calculator</title>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></s cript>
<style> body {
font-family: Arial, sans-serif; text-align: center;
margin-top: 50px;
}
.calculator {
display: inline-block; border: 2px solid #444; border-radius: 8px; padding: 20px; background:
#f7f7f7;
}
.display { width: 200px; height: 40px;
margin-bottom: 10px; font-size: 20px; text-align: right; padding: 5px;
border: 1px solid #ccc;
border-radius: 4px; background: #fff;
}
.btn {
width: 45px; height: 45px; margin: 5px; font-size: 18px; cursor: pointer;
}
</style>
</head>
<body ng-controller="CalculatorController">
<div class="calculator">
<div class="display">{{ display }}</div>
<div>
<button class="btn" ng-click="press('7')">7</button>
<button class="btn" ng-click="press('8')">8</button>
```

```
<button class="btn" ng-click="press('9')">9</button>
<button class="btn" ng-click="press('/')">/</button>
</div>
<div>
<button class="btn" ng-click="press('4')">4</button>
<button class="btn" ng-click="press('5')">5</button>
<button class="btn" ng-click="press('6')">6</button>
<button class="btn" ng-click="press('*')">*</button>
</div>
<div>
<button class="btn" ng-click="press('1')">1</button>
<button class="btn" ng-click="press('2')">2</button>
<button class="btn" ng-click="press('3')">3</button>
<button class="btn" ng-click="press('-')">-</button>
</div>
<div>
<button class="btn" ng-click="press('0')">0</button>
<button class="btn" ng-click="press('.')">.</button>
<button class="btn" ng-click="calculate()">=</button>
<button class="btn" ng-click="press('+')">+</button>
</div>
<div>
<button class="btn" ng-click="clear()">C</button>
</div>
</div>
<script> angular.module("calculatorApp", [])
.controller("CalculatorController", function($scope) {
```

OUTPUT:



COE (30)	
RECORD (10)	
OBS (10)	
VIVA (10)	
TOTAL (60)	

RESULT: -

Thus, the implementation of simple calculator using angular js is successfully completed.

Exp NO: 12

Date:

WEATHER APP USING MEAN

AIM: -

Demonstrate a weather app that display weather conditions, setup angular for frontend and fetch weather data using external API.Design a mongoose schema for city, temperature and humidity display.

ALGORITHM: -

Step 1: Project Setup

1. Initialize **Node.js + Express backend**.
    - npm init -y
    - Install dependencies: npm install express mongoose axios cors
2. Initialize **Angular frontend**.
    - ng new weather-app
    - Install HTTP client: ng add @angular/common/http

Step 2: Database Schema (Mongoose)

1. Connect to **MongoDB** in backend (mongoose.connect).
2. Define schema Weather with fields:
    - city: String
    - temperature: Number
    - humidity: Number
    - date: { type: Date, default: Date.now }
3. Export the model.

Step 3: External API Integration

1. Choose a weather API (e.g., **OpenWeatherMap**).
2. Install axios in backend to fetch API data.

3. Create a backend route /api/weather/:city that:
    - Calls the external weather API,
    - Extracts temperature and humidity,
    - Saves data to MongoDB,
    - Returns response to Angular.

Step 4: Angular Service (Frontend)

1. Create weather.service.ts.
2. Use HttpClient to call backend /api/weather/:city.
3. Return the JSON response.

Step 5: Angular Component

1. Create weather.component.ts and template weather.component.html.
2. Input field: enter city name.
3. Button: fetch weather.
4. Display results:
    - City name
    - Temperature
    - Humidity

Step 6: Display Data

1. Show results in a styled card (Angular Material optional).
2. Bind values with {{ city }}, {{ temperature }}, {{ humidity }}.
3. Handle errors (e.g., invalid city).

Step 7: Testing

1. Run backend: node server.js.
2. Run Angular frontend: ng serve.
3. Enter a city → Fetch data → Display weather.

CODE: -

SERVER.JS

```
const express = require("express");
const mongoose = require("mongoose");
const axios = require("axios");
const cors = require("cors");
const Weather = require("./models/Weather");

const app = express();
app.use(cors());
app.use(express.json());

// MongoDB connection
mongoose.connect("mongodb://127.0.0.1:27017/weatherDB", {
  useNewUrlParser: true,
  useUnifiedTopology: true,
});

// API route to fetch weather
app.get("/api/weather/:city", async (req, res) => {
  const city = req.params.city;
  const API_KEY = "YOUR_OPENWEATHER_API_KEY"; // Replace with your
OpenWeather key

  try {
    // Fetch from OpenWeatherMap API
    const response = await axios.get(

`https://api.openweathermap.org/data/2.5/weather?q=${city}&appid=${API_KEY}&units=m
etric`
    );
```

```
    const weatherData = {
      city: response.data.name,
      temperature: response.data.main.temp,
      humidity: response.data.main.humidity,
    };

    // Save to MongoDB
    const weather = new Weather(weatherData);
    await weather.save();

    res.json(weatherData);
  } catch (error) {
    res.status(500).json({ message: "Error fetching weather", error });
  }
});

// Start server
app.listen(5000, () => {
  console.log("Server running on http://localhost:5000");
}
);
```

Index.js

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Weather App</title>
<link rel="stylesheet" href="/css/style.css">
<link href="https://fonts.googleapis.com/css2?family=Open+Sans:wght@300;400;600&display=swap" rel="stylesheet">
</head>
<body>
<div class="container">
<div class="weather-card">
<h1>Weather Checker</h1>
<form action="/" method="post">
<input name="city" type="text" class="input-field" placeholder="Enter a City" required>
<button type="submit" class="btn">Get Weather</button>
</form>
<% if(weather){ %>
<p class="weather-text"><%= weather %></p>
<% } %>
<% if(error){ %>
<p class="error-text"><%= error %></p>
<% } %>
</div>
</div>
</body>
</html>
```

COE (30)	
RECORD (10)	
OBS (10)	
VIVA (10)	
TOTAL (60)	

RESULT: -

Thus, the implementation of web app using MEAN by integrating the API is successfully executed

Exp No: 13

Date:

HEALTHCARE WEBSITE USING MEAN

AIM: -

To Apply MEAN stack architecture to develop healthcare website. Create a REST API to register patients with name, age, contact number. Store in mongoDB using Mongoose models. Develop a module that includes doctor profiles via Express API. Display Doctor data on Angular front-end using server and component.

ALGORITHM: -

Step 1: Project Setup

1. Initialize Node.js + Express backend (npm init -y).
2. Install required dependencies:
3. npm install express mongoose cors body-parser
4. Setup Angular frontend using:
5. ng new healthcare-app

Step 2: Database Design (MongoDB + Mongoose)

1. Connect to MongoDB (mongoose.connect).
2. Define Patient Schema:
    - name (String)
    - age (Number)
    - contact (String)
3. Define Doctor Schema:
    - name (String)
    - specialization (String)
    - experience (Number)

Step 3: REST API with Express

1. Create routes for patients:
    - POST /api/patients → Register new patient.
    - GET /api/patients → Fetch all patients.

2. Create routes for doctors:
    - POST /api/doctors → Add new doctor profile.
    - GET /api/doctors → Fetch all doctor profiles.

Step 4: Angular Services

1. Create patient.service.ts:
    - Method to call POST /api/patients for registration.
    - Method to call GET /api/patients to display patients.
2. Create doctor.service.ts:
    - Method to call GET /api/doctors for doctor profiles.

Step 5: Angular Components

1. Patient Registration Component:
    - Form with input fields: name, age, contact.
    - Submit → Calls patient.service → Stores data in DB.
2. Doctor Display Component:
    - Fetch doctors from backend.
    - Display name, specialization, experience.

Step 6: Integration

1. Run backend (node server.js).
2. Run Angular (ng serve).
3. Ensure CORS is enabled in backend (app.use(cors())).
4. Connect Angular services to backend endpoints.

Step 7: Testing

1. Register a new patient → Data stored in MongoDB.
2. Add doctor profiles via API → Data saved in MongoDB.
3. Angular frontend displays:
    - Registered patients list.
    - Available doctors list.

CODE: -

server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const Patient = require("./models/Patient");
const Doctor = require("./models/Doctor");

const app = express();
app.use(cors());
app.use(express.json());

// MongoDB connection
mongoose.connect("mongodb://127.0.0.1:27017/healthcareDB", {
  useNewUrlParser: true,
  useUnifiedTopology: true,
});

// -------- Patient Routes --------
app.post("/api/patients", async (req, res) => {
  try {
    const patient = new Patient(req.body);
    await patient.save();
    res.status(201).json(patient);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

app.get("/api/patients", async (req, res) => {
  const patients = await Patient.find();
  res.json(patients);
});
```

```
// -------- Doctor Routes --------
app.post("/api/doctors", async (req, res) => {
  try {
    const doctor = new Doctor(req.body);
    await doctor.save();
    res.status(201).json(doctor);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

app.get("/api/doctors", async (req, res) => {
  const doctors = await Doctor.find();
  res.json(doctors);
});

// Start server
app.listen(5000, () => console.log("Server running on http://localhost:5000"));
```

Models: -

models/Patient.js

```
const mongoose = require("mongoose");
const PatientSchema = new mongoose.Schema({
  name: { type: String, required: true },
  age: { type: Number, required: true },
  contact: { type: String, required: true }
});

module.exports = mongoose.model("Patient", PatientSchema);
```

models/Doctor.js

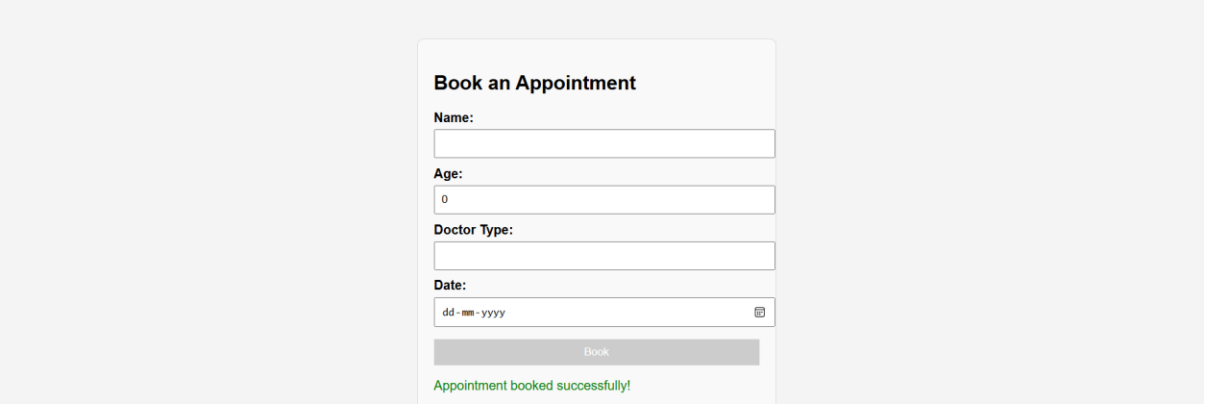
```
const mongoose = require("mongoose");
const DoctorSchema = new mongoose.Schema({
  name: { type: String, required: true },
```

```
  specialization: { type: String, required: true },
  experience: { type: Number, required: true }
});

module.exports = mongoose.model("Doctor", DoctorSchema);
```

OUTPUT: -







COE (30)	
RECORD (10)	
OBS (10)	
VIVA (10)	
TOTAL (60)	

RESULT: -

Thus, the implementation of healthcare website is successfully executed using MEAN stack.

Exp No: 14

Date:

TRAVEL BOOKING WEBSITE USING MEAN

AIM: -

To develop a travel booking website using MEAN stack architecture.

ALGORITHM: -

Step 1: Project Setup

1. Initialize Node.js + Express backend (npm init -y).
2. Install dependencies: npm install express mongoose cors body-parser.
3. Initialize Angular frontend: ng new travel-booking-app.
4. Enable CORS for backend API.

Step 2: Database Design (MongoDB + Mongoose)

1. Connect to MongoDB using mongoose.connect.
2. Define User Schema:
    - name (String)
    - email (String, unique)
    - password (String, hashed)
3. Define Travel Package Schema:
    - destination (String)
    - price (Number)
    - startDate (Date)
    - endDate (Date)
    - availableSeats (Number)
4. Define Booking Schema:
    - userId (ObjectId ref User)
    - packageId (ObjectId ref TravelPackage)
    - bookingDate (Date)

Step 3: REST API Development (Express)

1. User Routes:
    - POST /api/users/register → Register user.

- POST /api/users/login → User login.

2. Travel Package Routes:
   - POST /api/packages → Add new travel package.
   - GET /api/packages → Get all packages.
3. Booking Routes:
   - POST /api/bookings → Book a travel package.
   - GET /api/bookings/:userId → Fetch user's bookings.

Step 4: Angular Services

1. User Service:
   - Handles register & login.
2. Package Service:
   - Fetch all travel packages.
   - Add new travel packages (admin only).
3. Booking Service:
   - Book a package.
   - Fetch booking history.

Step 5: Angular Components

1. User Authentication Component
   - Register form (name, email, password).
   - Login form.
2. Package Listing Component
   - Show all available travel packages.
   - Button to "Book Now".
3. Booking Component
   - Shows user's booked trips.

Step 6: Integration

1. Connect Angular services to Express APIs.
2. Use HttpClientModule in Angular for API calls.
3. Implement JWT authentication (optional for login security).

Step 7: Testing

1. Register a user → stored in MongoDB.

2. Add travel packages → visible on Angular UI.
3. Book a package → stored in MongoDB.
4. Retrieve and display user's booking history.

Code: -

Mongoose Models: -

```
const mongoose = require('mongoose');

const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({

  email: { type: String, required: true, unique: true },

  password: { type: String, required: true },

  name: { type: String, required: true }

});

userSchema.pre('save', async function(next) {

  if (!this.isModified('password')) return next();

  this.password = await bcrypt.hash(this.password, 10);

  next();});

userSchema.methods.comparePassword = async function(password) {

  return bcrypt.compare(password, this.password);};

module.exports = mongoose.model('User ', userSchema);
```

Data connection: -

```
const mongoose = require('mongoose');

require('dotenv').config();

mongoose.connect(process.env.MONGODB_URI, {

useNewUrlParser: true,

  useUnifiedTopology: true

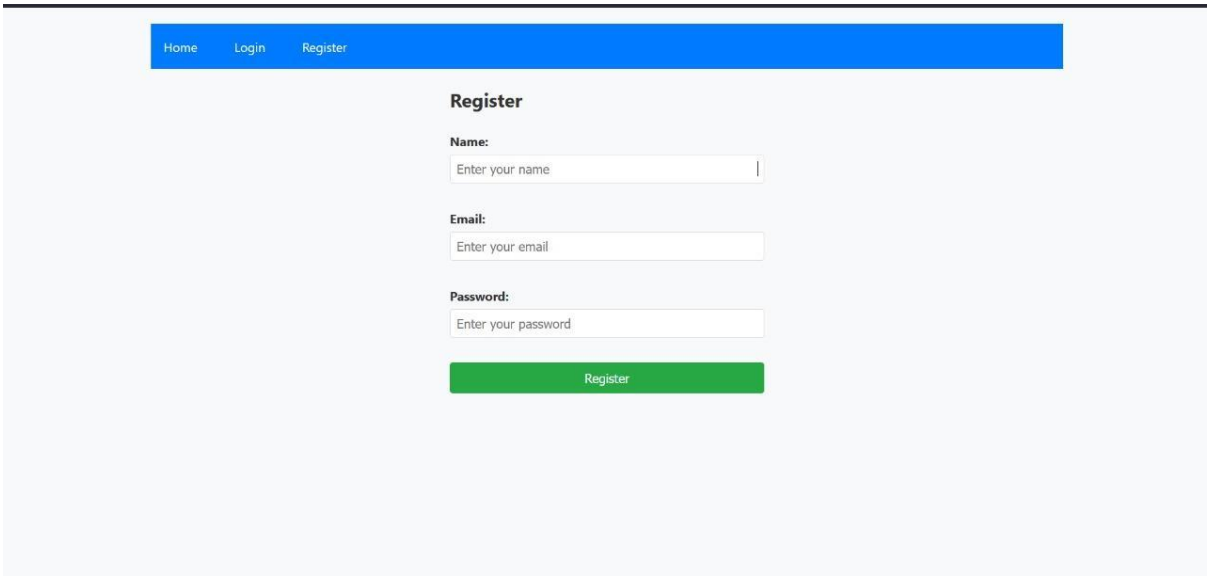
})

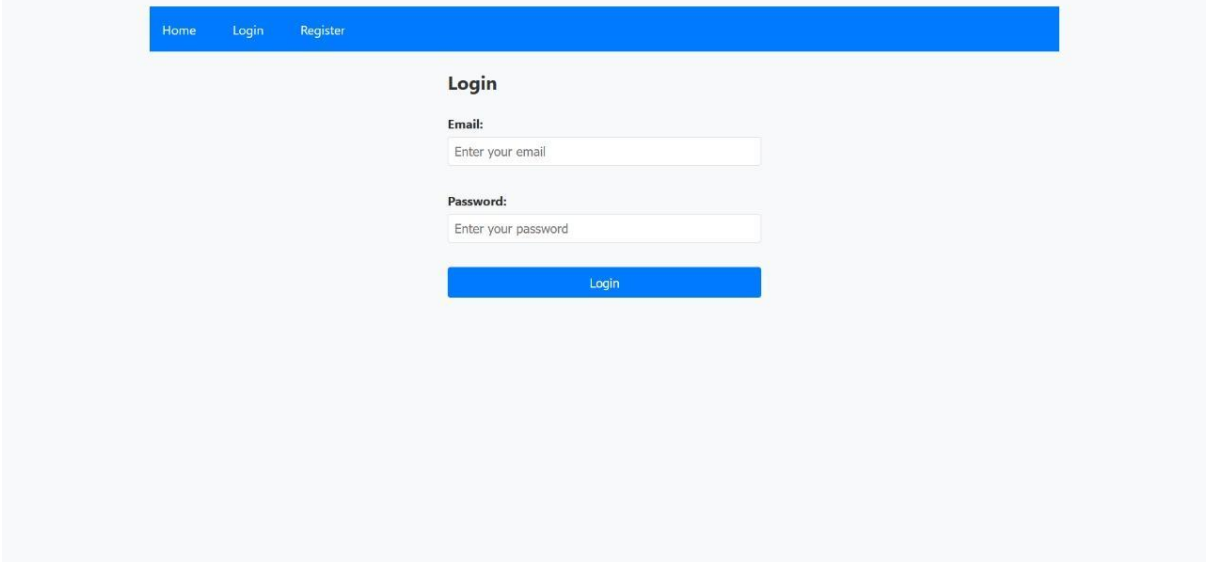
.then(() => console.log('MongoDB connected'))

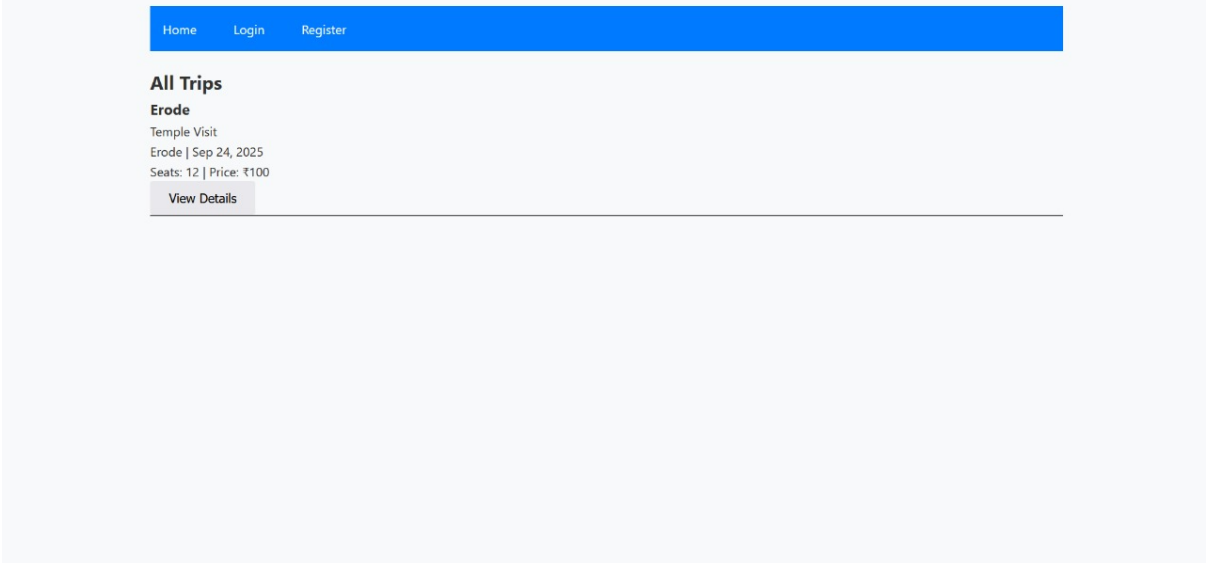
.catch(err => console.error('MongoDB connection error:', err));

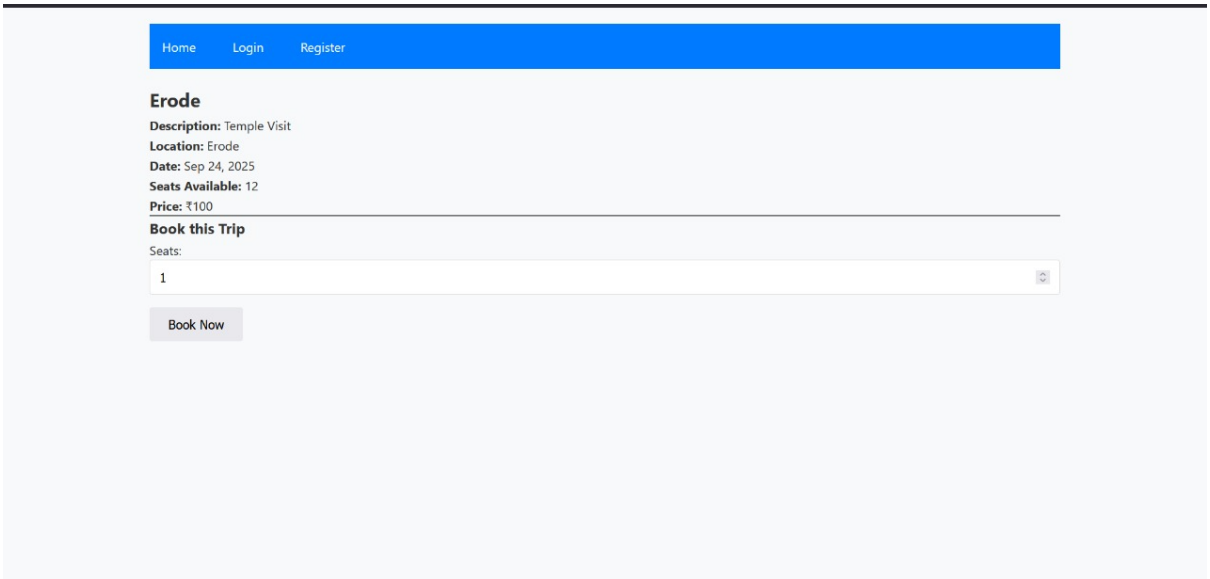
module.exports = mongoose;
```

OUTPUT: -




Home
Login
Register
Login
Email:
Enter your email
Password:
Enter your password
Login


Home
Login
Register
All Trips
Erode
Temple Visit
Erode | Sep 24, 2025
Seats: 12 | Price: ₹100
View Details



COE (30)	
RECORD (10)	
OBS (10)	
VIVA (10)	
TOTAL (60)	

RESULT: -

Thus, the implementation of a travel booking website using MEAN stack architecture is successfully executed.

Exp No: 15

Date:

BUILD AN E-COMMERCE WEBSITE THAT DISPLAY PRODUCT LISTINGS WITH PAYMENT MODULE.

AIM: -

To build an e-commerce website that display product listings with payment module using razor pay API.

ALGORITHM: -

STEP 1: Start.

STEP 2: Gather requirements for the website.

- Define target users (customers, admins).
- List features (product catalog, cart, checkout, payment).
- Identify supported payment gateways.

STEP 3: Plan the system architecture.

- Choose frontend technology (HTML, CSS, JavaScript, React/Angular).
- Choose backend technology (PHP, Node.js, Django, etc.).
- Select database (MySQL, MongoDB, PostgreSQL).

STEP 4: Design the database schema.

- Create tables for Users (user_id, name, email, password).
- Create tables for Products (product_id, name, description, price, stock, image).
- Create tables for Cart (cart_id, user_id, product_id, quantity).
- Create tables for Orders (order_id, user_id, product details, payment status).
- Create tables for Payments (payment_id, order_id, amount, transaction_id, status).

STEP 5: Design the user interface (UI).

- Homepage with navigation bar and product highlights.
- Product listing page with search and filters.
- Product detail page with description and “Add to Cart” option.

- Shopping cart page showing selected products and total cost.
- Checkout page with billing, shipping, and payment form.
- Admin dashboard to manage products and orders.

STEP 6: Implement user authentication.

- Registration module (username, email, password).
- Login/logout functionality with secure sessions.
- Password encryption for security.

STEP 7: Develop product listing module.

- Fetch products from database.
- Display with image, name, price, and “Add to Cart” button.
- Implement sorting and filtering (price, category, rating).

STEP 8: Develop shopping cart functionality.

- Add items to cart.
- Update quantities or remove items.
- Calculate total cost dynamically.

STEP 9: Implement checkout process.

- Collect shipping and billing information.
- Generate order summary before payment.

STEP 10: Integrate payment gateway module.

- Connect with API of payment provider (PayPal, Stripe, Razorpay, etc.).
- Securely process payment.
- Update order status (paid, pending, failed).

STEP 11: Implement order management system.

- Store confirmed order in database.
- Send confirmation email/notification to customer.
- Update product stock after successful payment.

STEP 12: Test the website.

- Test functionality (UI, cart, checkout, payment).
- Test performance (loading speed, database queries).
- Test security (SQL injection, XSS, secure payments).

STEP 13: Deploy website on hosting server.

- Set up domain and hosting.
- Configure database on server.
- Ensure SSL (HTTPS) for secure transactions.

STEP 14: Maintain and update system.

- Monitor transactions and fix bugs.
- Update product listings and prices.
- Add new features as required.

STEP 15: End.

CODE: -

Server.js

```
const express = require("express");
const bodyParser = require("body-parser");
const stripe = require("stripe")("sk_test_your_secret_key"); // test secret key

const app = express();
app.set("view engine", "ejs");
app.use(bodyParser.urlencoded({ extended: true }));

// Sample product list
const products = [
  { id: 1, name: "Laptop", price: 80000 },
  { id: 2, name: "Smartphone", price: 30000 },
  { id: 3, name: "Headphones", price: 2000 },
];

// Homepage – product listings
app.get("/", (req, res) => {
  res.render("index", { products });
});
```

```
// Checkout page
app.get("/checkout/:id", (req, res) => {
  const product = products.find(p => p.id == req.params.id);

  res.render("checkout", { product });
});

// Handle payment
app.post("/payment", async (req, res) => {
  const { productId } = req.body;
  const product = products.find(p => p.id == productId);

  try {
    const paymentIntent = await stripe.paymentIntents.create({
      amount: product.price,
      currency: "inr",
      payment_method_types: ["card"],
    });

    res.send(`
      <h2>Payment Successful!</h2>
      <p>You purchased: ${product.name}</p>
      <p>Amount Paid: ₹${product.price}</p>
    `);
  } catch (error) {
    res.send("Payment Failed: " + error.message);
  }
});

app.listen(3000, () => console.log("Server running at http://localhost:3000"));
```

views/index.ejs

```
<!DOCTYPE html>
<html>
<head>
```

```
  <title>E-commerce Store</title>
 </head>
 <body>

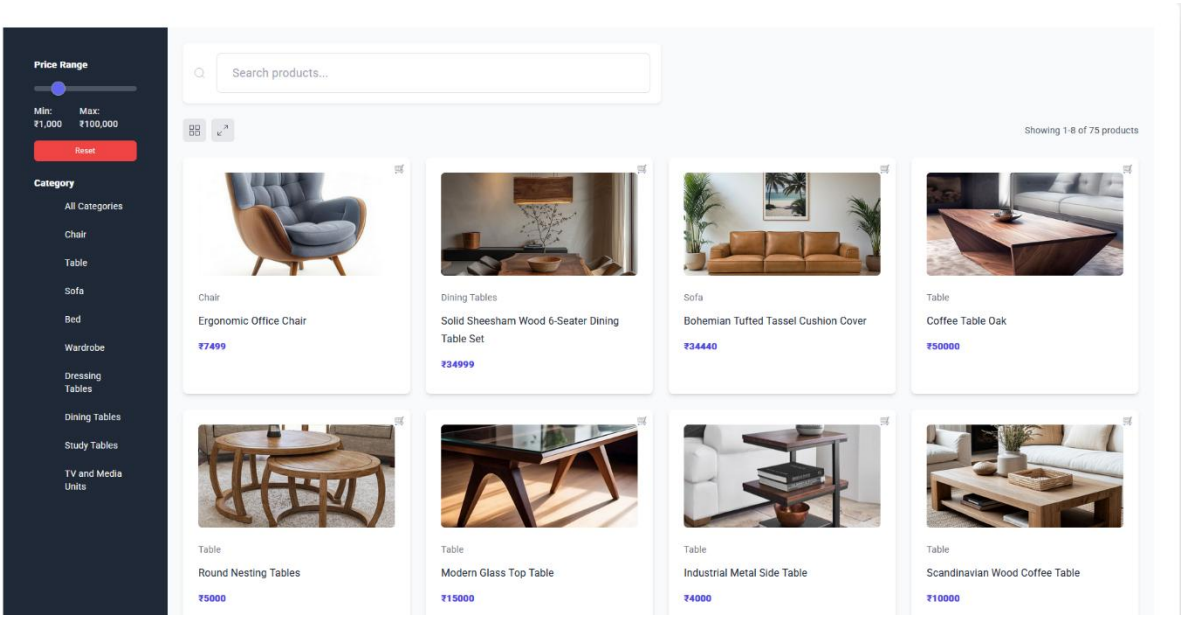

  <h1>Product Listings</h1>
  <ul>
   <% products.forEach(product => { %>
    <li>
     <%= product.name %> - ₹<%= product.price %>
     <a href="/checkout/<%= product.id %>">Buy Now</a>
    </li>
   <% }) %>
  </ul>
 </body>
 </html>
```

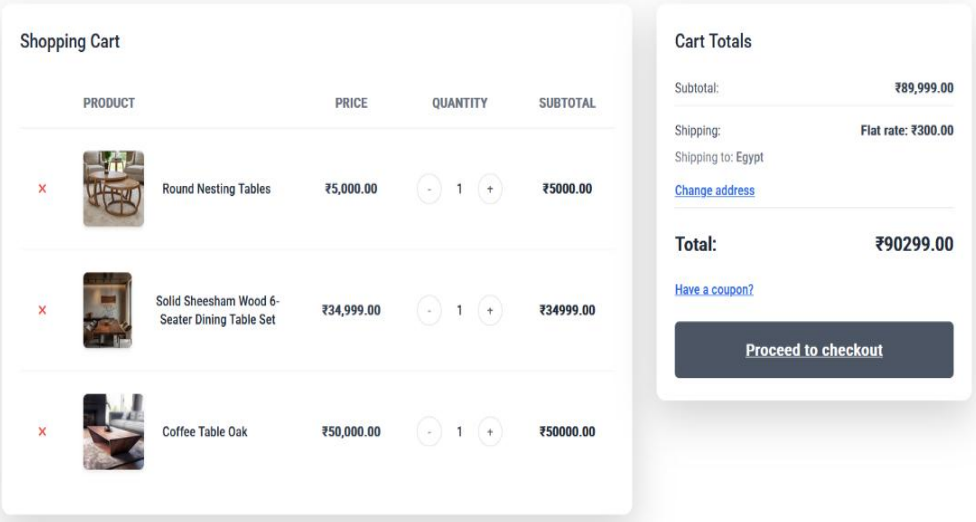
views/checkout.ejs

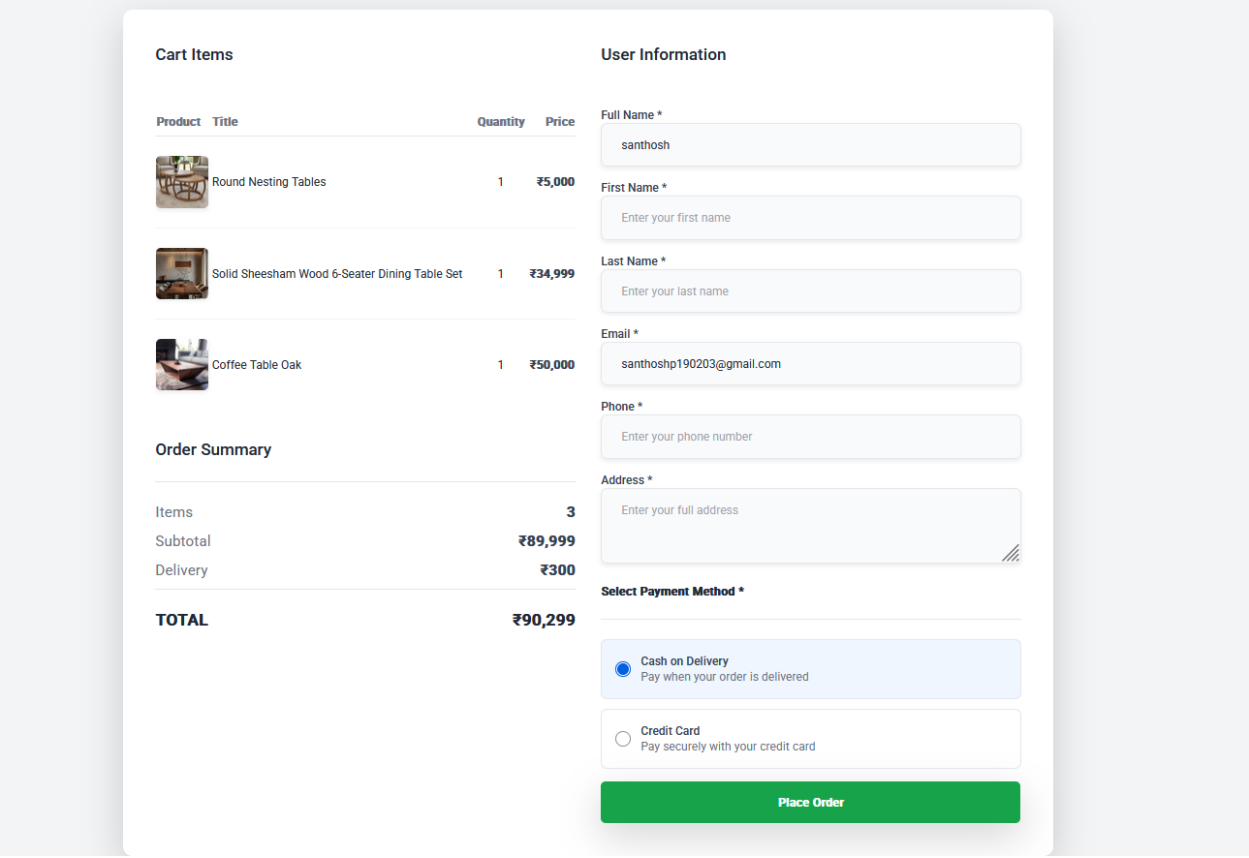
```
<!DOCTYPE html>
<html>
<head>
  <title>Checkout</title>
</head>
<body>
  <h1>Checkout</h1>
  <p>Product: <%= product.name %></p>
  <p>Price: ₹<%= product.price %></p>

  <form action="/payment" method="POST">
    <input type="hidden" name="productId" value="<%= product.id %>">
    <button type="submit">Pay with Stripe (Test Mode)</button>
  </form>
</body>
</html
```

OUTPUT: -


Price Range
Min: ₹1,000
Max: ₹100,000
Reset
Category
All Categories
Chair
Table
Sofa
Bed
Wardrobe
Dressing Tables
Dining Tables
Study Tables
TV and Media Units
Search products...
Showing 1-8 of 75 products
Chair
Ergonomic Office Chair
₹7499
Dining Tables
Solid Sheesham Wood 6-Seater Dining Table Set
₹34999
Sofa
Bohemian Tufted Tassel Cushion Cover
₹34440
Table
Coffee Table Oak
₹50000
Table
Round Nesting Tables
₹5000
Table
Modern Glass Top Table
₹15000
Table
Industrial Metal Side Table
₹4000
Table
Scandinavian Wood Coffee Table
₹10000


Shopping Cart
PRODUCT
PRICE
QUANTITY
SUBTOTAL
Round Nesting Tables
₹5,000.00
1
₹5000.00
Solid Sheesham Wood 6-Seater Dining Table Set
₹34,999.00
1
₹34999.00
Coffee Table Oak
₹50,000.00
1
₹50000.00
Cart Totals
Subtotal:
₹89,999.00
Shipping:
Flat rate: ₹300.00
Shipping to: Egypt
Change address
Total:
₹90299.00
Have a coupon?
Proceed to checkout



COE (30)	
RECORD (10)	
OBS (10)	
VIVA (10)	
TOTAL (60)	

Result: -

Thus, the implementation of payment module with the product in the e-commerce website is successfully executed.